Purpose

CAN-0 — canonical/ Purpose + boundaries

Piece A — Purpose

File: `Axion/libraries/canonical/CAN-0_purpose.md`

---
library: canonical
id: CAN-0
schema_version: 1.0.0
status: draft

CAN-0 — canonical/ Purpose + Boundaries

Purpose
`canonical/` defines the **Canonical Spec contract**: the single source of truth built from normalized intake + resolved standards. It provides:
- a consistent, normalized representation of the target system
- stable entity identifiers and relationships
- reference integrity rules
- a formal model for unknowns/assumptions
- constraints that downstream stages rely on (templates, planning, verification)

What it governs (in scope)
- Canonical Spec entity model (what entities exist, minimum fields)
- Relationship model (how entities reference each other)
- ID rules (stable, deterministic ID generation and normalization)
- Reference integrity rules (no dangling refs, consistent naming)
- Unknowns/assumptions artifact model (captured explicitly)
- Canonical build invariants and validation rules

What it does NOT govern (out of scope)
- Intake forms and normalization rules → `intake/`
- Which standards apply and how they're resolved → `standards/`
- Gate DSL semantics → `gates/`
- Pipeline ordering, stage reports, run manifest → `orchestration/`
- Template registry/selection/render rules → `templates/`
- Planning artifacts (WBS, acceptance map) → `planning/`
- Proof ledger and verification rules → `verification/`
- Workspace/pins/adapters/quotas → `system/`
- Risk classes and overrides → `policy/`

- Knowledge items (library content) → `knowledge/`

Consumers
- Template selection (S5)
- Planning (S6)
- Template fill (S7)
- Gate evaluation (S8) for coverage/integrity checks
- Kit packaging (S9) for final deliverable completeness

Determinism requirements
- Canonical spec is a pure function of:
  - NORMALIZED_INPUT
  - STANDARDS_SNAPSHOT
  - canonical build rules (this library, pinned)
- IDs are stable and reproducible.
- Unknowns are explicit and do not hide inside free text.

Piece B — Boundary checklist

File: `Axion/libraries/canonical/CAN-0_boundary_checklist.md`

---
library: canonical
id: CAN-0a
schema_version: 1.0.0
status: draft

CAN-0a — Boundary Checklist

Belongs in `canonical/` if it answers:
- “What is the canonical representation of the target system?”
- “How are entities identified and related?”
- “What is an unknown/assumption and how is it recorded?”
- “What integrity rules must the canonical spec satisfy?”

Does NOT belong in `canonical/` if it answers:
- “What are the intake questions?” (intake)
- “Which standards apply?” (standards)
- “How do we render templates?” (templates)
- “How do we evaluate gates?” (gates)

Canonical Spec entity model

CAN-1 — Canonical Spec entity model (entities + relationships)

Piece A — Canonical entity model (concept)

File: `Axion/libraries/canonical/CAN-1_entity_model.md`

---
library: canonical
id: CAN-1
schema_version: 1.0.0
status: draft

CAN-1 — Canonical Spec Entity Model

Purpose
Define the minimum canonical entities and relationship structures that downstream stages can rely on.

Canonical Spec = entity graph
The canonical spec is a graph of entities with stable IDs and typed relationships.

Minimum entity types (baseline)
- project
- domain
- capability
- component
- data_entity
- endpoint
- integration
- workflow
- requirement
- constraint
- risk

You can expand this list later, but these form a stable minimum set for planning + templates.

Relationship types (baseline)
- contains (parent → child)
- depends_on (entity → entity)
- exposes (component → endpoint)

- stores (component → data_entity)
- integrates_with (component → integration)
- satisfies (component/workflow → requirement)
- constrained_by (any → constraint)

Piece B — Canonical spec schema (JSON Schema)

File: `Axion/libraries/canonical/schemas/canonical_spec.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "canonical_spec.v1",
  "type": "object",
  "required": ["spec_id", "version", "entities", "relationships", "created_at", "owner"],
  "properties": {
    "spec_id": { "type": "string", "pattern": "^CAN-[A-Z0-9]{6,}$" },
    "version": { "type": "string", "minLength": 1 },

    "entities": {
      "type": "array",
      "minItems": 1,
      "items": { "$ref": "#/$defs/entity" }
    },

    "relationships": {
      "type": "array",
      "items": { "$ref": "#/$defs/relationship" }
    },

    "meta": {
      "type": "object",
      "description": "Optional run-scoped metadata (profile, stack hints, etc.)",
      "additionalProperties": true
    },

    "created_at": { "type": "string", "format": "date-time" },
    "owner": { "type": "string", "minLength": 2 }
  },

  "$defs": {
    "entity": {
```

```
  "type": "object",
  "required": ["entity_id", "type", "name"],
  "properties": {
    "entity_id": { "type": "string", "pattern": "^E-[A-Z0-9]{6,}$" },
    "type": {
      "type": "string",
      "enum": [
        "project",
        "domain",
        "capability",
        "component",
        "data_entity",
        "endpoint",
        "integration",
        "workflow",
        "requirement",
        "constraint",
        "risk"
      ]
    },
    "name": { "type": "string", "minLength": 2 },

    "attributes": {
      "type": "object",
      "description": "Type-specific fields (kept flexible but bounded by contracts in CAN-2/3)",
      "additionalProperties": true
    }
  },
  "additionalProperties": false
},

"relationship": {
  "type": "object",
  "required": ["rel_id", "type", "from", "to"],
  "properties": {
    "rel_id": { "type": "string", "pattern": "^R-[A-Z0-9]{6,}$" },
    "type": {
      "type": "string",
      "enum": [
        "contains",
        "depends_on",
        "exposes",
        "stores",
        "integrates_with",
```

```
          "satisfies",
          "constrained_by"
        ]
      },
      "from": { "type": "string", "pattern": "^E-[A-Z0-9]{6,}$" },
      "to": { "type": "string", "pattern": "^E-[A-Z0-9]{6,}$" },

      "attributes": {
        "type": "object",
        "additionalProperties": true
      }
    },
    "additionalProperties": false
  }
},

"additionalProperties": false
}
```

Piece C — Determinism rules (graph stability)

File: `Axion/libraries/canonical/CAN-1_determinism_rules.md`

```
---
library: canonical
id: CAN-1a
schema_version: 1.0.0
status: draft
---

# CAN-1a — Determinism Rules (Canonical Graph)

- entities are sorted deterministically in storage:
  - primary sort: type
  - secondary sort: name
  - tertiary sort: entity_id
- relationships are sorted deterministically:
  - type, from, to, rel_id
- Any derived entity/relationship must be reproducible from pinned inputs + rules.
```

Piece D — Validation checklist

File: `Axion/libraries/canonical/CAN-1_validation_checklist.md`

---
library: canonical
id: CAN-1b
schema_version: 1.0.0
status: draft

CAN-1b — Validation Checklist

- [ ] canonical_spec validates against schema
- [ ] entity_id values are unique
- [ ] relationship rel_id values are unique
- [ ] relationship from/to references exist in entities (strict mode)
- [ ] entity and relationship arrays are deterministically ordered (if enforced)

Say **“next”** for **CAN-2 ID rules (stable IDs, normalization, dedupe)**.

ID rules

CAN-2 — ID rules (stable IDs, normalization, dedupe)

Piece A — ID strategy (concept)

File: `Axion/libraries/canonical/CAN-2_id_rules.md`

---
library: canonical
id: CAN-2
schema_version: 1.0.0
status: draft

CAN-2 — ID Rules (Stable IDs + Dedupe)

Purpose
Ensure every entity and relationship has a stable, reproducible ID so:
- planning/templates can reference entities reliably
- reruns/resumes preserve identity
- diffs are meaningful (not re-ID noise)

ID types
- entity_id: E-XXXXXX…
- rel_id: R-XXXXXX…
- spec_id: CAN-XXXXXX…

Requirements
- IDs are stable across reruns given the same inputs.
- IDs are generated deterministically from canonical keys, not random.
- Dedupe rules prevent duplicate entities when the same concept appears multiple times in intake.

Canonical key (entity identity)
Each entity must have a canonical key, for example:
- project: `project:<project_slug>`
- domain: `domain:<domain_slug>`
- endpoint: `endpoint:<method>:<path>`
- data_entity: `data_entity:<name_slug>`
- component: `component:<name_slug>`
- integration: `integration:<provider_slug>:<purpose_slug>`

Piece B — Deterministic ID generation spec

File: `Axion/libraries/canonical/CAN-2_id_generation_spec.md`

```
---
library: canonical
id: CAN-2a
schema_version: 1.0.0
status: draft
---

# CAN-2a — Deterministic ID Generation

## Inputs
- canonical_key (string)
- namespace salt (workspace_id + project_id optional)
- versioned id algorithm (id_algo_version)

## Algorithm (concept)
1) Normalize canonical_key:
   - lowercase
   - trim
   - collapse whitespace
2) Compute hash:
   - hash = sha256(id_algo_version + ":" + namespace + ":" + canonical_key)
3) Encode:
   - take first N chars of base32/base16
4) Prefix:
   - entity_id: `E-<encoded>`
   - rel_id: `R-<encoded>`

## Notes
- The algorithm version must be pinned and recorded in canonical spec meta.
- Namespace inclusion:
  - If you want IDs stable across projects, omit namespace.
  - If you want isolation, include workspace/project namespace.
Pick one and keep it consistent.
```

Piece C — Dedupe rules (entity merge policy)

File: `Axion/libraries/canonical/CAN-2_dedupe_rules.md`

---
library: canonical
id: CAN-2b
schema_version: 1.0.0
status: draft

CAN-2b — Dedupe Rules

Purpose
Prevent accidental duplicates when multiple intake fields map to the same entity.

Dedupe strategy
- Compute canonical_key for each candidate entity.
- If canonical_key already exists:
  - merge attributes (non-conflicting)
  - if conflicting values:
    - prefer explicitly provided intake values over derived defaults
    - record conflict into UNKNOWN_ASSUMPTIONS as an "assumption needed" item

Merge rules
- Scalars:
  - if same → keep
  - if different → conflict
- Lists:
  - union + stable sort
- Objects:
  - shallow merge; conflicts recorded

No silent overwrites
If conflict occurs, it must be recorded as an unknown/assumption item.

Piece D — Machine-readable ID rules registry (optional)

File: `Axion/libraries/canonical/registries/id_rules.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "id_rules.v1",
  "id_algo_version": "1",
  "namespace_mode": "workspace_project",
  "entity_prefix": "E-",
```

```
  "relationship_prefix": "R-",
  "canonical_key_templates": {
    "project": "project:{project_slug}",
    "domain": "domain:{domain_slug}",
    "component": "component:{name_slug}",
    "endpoint": "endpoint:{method}:{path}",
    "data_entity": "data_entity:{name_slug}",
    "integration": "integration:{provider_slug}:{purpose_slug}"
  }
}
```

Piece E — Determinism rules

File: `Axion/libraries/canonical/CAN-2_determinism_rules.md`

```
---
library: canonical
id: CAN-2c
schema_version: 1.0.0
status: draft
---

# CAN-2c — Determinism Rules (IDs)

- ID generation uses canonical_key + pinned id_algo_version.
- Namespace mode is fixed (workspace_project or global) and pinned.
- Dedupe is canonical_key-based and stable.
- Conflicts never silently overwrite; they become unknown/assumption records.
```

Piece F — Validation checklist

File: `Axion/libraries/canonical/CAN-2_validation_checklist.md`

```
---
library: canonical
id: CAN-2d
schema_version: 1.0.0
status: draft
---
```

CAN-2d — Validation Checklist

- [ ] id_rules registry exists (if used) and is pinned
- [ ] all entities have deterministic canonical_key generation
- [ ] entity_id and rel_id are deterministic and stable on rerun
- [ ] dedupe merges do not silently overwrite conflicts
- [ ] conflicts create UNKNOWN_ASSUMPTIONS entries

Reference integrity rules

CAN-3 — Reference integrity rules (foreign refs, missing refs)

Piece A — Reference integrity model (concept)

File: `Axion/libraries/canonical/CAN-3_reference_integrity.md`

```
---
library: canonical
id: CAN-3
schema_version: 1.0.0
status: draft
---

# CAN-3 — Reference Integrity Rules

## Purpose
Ensure the canonical spec graph is internally consistent:
- no dangling references
- relationship endpoints exist
- entity attributes that reference other entities use valid IDs
- consistent tenancy scoping if you include it

## Integrity surfaces
1) Relationship integrity (from/to exist)
2) Attribute reference integrity (attributes contain valid entity_id references)
3) Type compatibility (relationship type matches entity types)
4) Uniqueness constraints (no duplicate endpoints, etc.)
```

Piece B — Allowed relationship type constraints (type compatibility)

File: `Axion/libraries/canonical/registries/relationship_constraints.v1.json`

```
{
  "schema_version": "1.0.0",
  "registry_id": "relationship_constraints.v1",
  "constraints": [
    { "type": "contains", "from_types": ["project", "domain", "capability", "component"], "to_types":
["domain", "capability", "component", "endpoint", "data_entity", "workflow"] },
```

```
    { "type": "depends_on", "from_types": ["component", "workflow", "integration"], "to_types":
["component", "integration", "data_entity"] },
    { "type": "exposes", "from_types": ["component"], "to_types": ["endpoint"] },
    { "type": "stores", "from_types": ["component"], "to_types": ["data_entity"] },
    { "type": "integrates_with", "from_types": ["component"], "to_types": ["integration"] },
    { "type": "satisfies", "from_types": ["component", "workflow"], "to_types": ["requirement"] },
    { "type": "constrained_by", "from_types": ["project", "domain", "component", "endpoint",
"workflow"], "to_types": ["constraint", "risk"] }
  ]
}
```

Piece C — Integrity checks (rules list)

File: `Axion/libraries/canonical/CAN-3_integrity_checks.md`

```
---
library: canonical
id: CAN-3a
schema_version: 1.0.0
status: draft
---

# CAN-3a — Integrity Checks

## CAN-INTEGRITY-01 Relationship endpoints exist
For every relationship:
- from must reference an existing entity_id
- to must reference an existing entity_id

## CAN-INTEGRITY-02 Relationship type compatibility
For every relationship:
- entity(from).type must be allowed for rel.type
- entity(to).type must be allowed for rel.type
(using relationship_constraints registry)

## CAN-INTEGRITY-03 Attribute refs are valid
If an entity attribute contains references like:
- owner_entity_id
- component_id
- data_entity_id
- endpoint_id
then each must reference a valid entity_id.
```

(Implementation: maintain a list of ref-like attribute keys or annotate schemas per entity type later.)

CAN-INTEGRITY-04 Uniqueness constraints
- endpoint canonical keys must be unique: (method + path)
- component name_slug must be unique within project scope
- data_entity name_slug must be unique within project scope

CAN-INTEGRITY-05 Unknowns recorded, not hidden
If an entity requires a field but it's missing/ambiguous:
- do not invent silently
- record as UNKNOWN_ASSUMPTIONS entry

Piece D — Determinism rules

File: `Axion/libraries/canonical/CAN-3_determinism_rules.md`

---
library: canonical
id: CAN-3b
schema_version: 1.0.0
status: draft

CAN-3b — Determinism Rules (Integrity)

- Integrity checks run in fixed order CAN-INTEGRITY-01..05.
- Failures are sorted deterministically (by entity_id / rel_id).
- Constraints registry is pinned via system pins.
- Unknowns are emitted as explicit unknown records, not inline notes.

Piece E — Validation checklist

File: `Axion/libraries/canonical/CAN-3_validation_checklist.md`

---
library: canonical
id: CAN-3c
schema_version: 1.0.0
status: draft

CAN-3c — Validation Checklist

- [ ] relationship_constraints registry exists and validates
- [ ] all relationships reference existing entity IDs
- [ ] relationship type compatibility enforced
- [ ] uniqueness constraints enforced (endpoint/component/data_entity)
- [ ] missing/ambiguous required data results in UNKNOWN_ASSUMPTIONS entries

Unknowns mode

CAN-4 — Unknowns / assumptions model (explicit, enforceable)

Piece A — Unknowns model (concept)

File: `Axion/libraries/canonical/CAN-4_unknowns_assumptions_model.md`

---
library: canonical
id: CAN-4
schema_version: 1.0.0
status: draft

CAN-4 — Unknowns / Assumptions Model

Purpose
Capture uncertainty explicitly so the system does not:
- silently invent missing details
- bury assumptions in prose
- break determinism across reruns

Unknowns are first-class artifacts that can drive:
- gate failures (canonical integrity)
- template placeholders (mark as TBD)
- planning tasks (resolve unknowns)
- operator approvals (accept assumptions)

Definitions
- **Unknown**: a missing or ambiguous fact required to proceed correctly.
- **Assumption**: a provisional chosen value used to proceed, which must be recorded and reviewable.
- **Resolution**: a confirmed value replacing an unknown/assumption.

Where unknowns come from
- intake missing fields
- intake conflicts (two values)
- standards gaps (standard requires something not provided)
- canonical dedupe conflicts (CAN-2)

Piece B — Unknowns artifact schema

File:
`Axion/libraries/canonical/schemas/unknown_assumptions.v1.schema.json`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "unknown_assumptions.v1",
  "type": "object",
  "required": ["ua_id", "run_id", "unknowns", "assumptions", "generated_at"],
  "properties": {
    "ua_id": { "type": "string", "pattern": "^UA-[A-Z0-9]{6,}$" },
    "run_id": { "type": "string" },

    "unknowns": {
      "type": "array",
      "items": { "$ref": "#/$defs/ua_item" }
    },

    "assumptions": {
      "type": "array",
      "items": { "$ref": "#/$defs/ua_item" }
    },

    "generated_at": { "type": "string", "format": "date-time" }
  },

  "$defs": {
    "ua_item": {
      "type": "object",
      "required": ["item_id", "kind", "topic", "pointer", "severity", "prompt"],
      "properties": {
        "item_id": { "type": "string", "pattern": "^UAITEM-[A-Z0-9]{6,}$" },
        "kind": { "type": "string", "enum": ["unknown", "assumption"] },

        "topic": { "type": "string", "minLength": 3 },
        "pointer": { "type": "string", "description": "Pointer into canonical_spec or normalized_input
(JSON pointer)" },

        "severity": { "type": "string", "enum": ["low", "medium", "high", "blocking"] },

        "prompt": { "type": "string", "minLength": 5, "description": "Question to resolve the unknown"
},
```

```
      "proposed_value": { "description": "Present for assumptions", "type": ["string", "number",
"boolean", "object", "array", "null"] },

      "source": {
        "type": "object",
        "properties": {
          "from": { "type": "string", "enum": ["intake_missing", "intake_conflict", "standards_gap",
"dedupe_conflict", "builder_default"] },
          "refs": { "type": "array", "items": { "type": "string" } }
        },
        "additionalProperties": false
      }
    },
    "additionalProperties": false
  }
 },

 "additionalProperties": false
}
```

Piece C — Unknowns rules (when to create unknown vs assumption)

File: `Axion/libraries/canonical/CAN-4_rules.md`

```
---
library: canonical
id: CAN-4a
schema_version: 1.0.0
status: draft
---

# CAN-4a — Unknown/Assumption Rules

## Default behavior
- If required information is missing → create an **unknown**.
- If the system chooses a default to proceed → create an **assumption**.

## Blocking severity
Mark as severity=blocking when:
- it affects security boundaries (auth, access control)
- it affects data integrity (schema, migration, money)
```

- it affects compliance classification (PII/PHI/payment)
- it affects external integrations (webhooks, payment processors)

Non-blocking severity
Mark as low/medium when:
- it affects naming, styling, or optional features
- it affects non-critical performance defaults

No hidden assumptions
Every builder default used must be captured as an assumption item (even if low severity).

Piece D — Determinism rules

File: `Axion/libraries/canonical/CAN-4_determinism_rules.md`

---
library: canonical
id: CAN-4b
schema_version: 1.0.0
status: draft

CAN-4b — Determinism Rules (Unknowns)

- Unknowns/assumptions generation is deterministic from pinned inputs + rules.
- item_id generation follows the same deterministic ID principles (CAN-2 style) using:
  - kind + pointer + topic as canonical key.
- unknowns and assumptions lists are sorted by:
  - severity (blocking → high → medium → low)
  - pointer
  - item_id

Piece E — Validation checklist

File: `Axion/libraries/canonical/CAN-4_validation_checklist.md`

---
library: canonical
id: CAN-4c
schema_version: 1.0.0

status: draft

CAN-4c — Validation Checklist

- [ ] unknown_assumptions artifact validates against schema
- [ ] every item has pointer + severity + prompt
- [ ] builder defaults are recorded as assumptions (no silent defaults)
- [ ] ordering is deterministic (severity → pointer → id)

Canonical artifact schemas

CAN-5 — Canonical artifacts + reports (schemas + outputs)

Piece A — Canonical build outputs (what S4 must produce)

File: `Axion/libraries/canonical/CAN-5_artifacts.md`

---
library: canonical
id: CAN-5
schema_version: 1.0.0
status: draft

CAN-5 — Canonical Artifacts (S4 Outputs)

Purpose
Define the canonical-stage outputs that downstream stages require.

Required artifacts
S4_CANONICAL_BUILD must produce:

1) **CANONICAL_SPEC** (canonical_spec.v1)
- entity graph representing the target system
- stable IDs and relationships
- minimal metadata for downstream selection/planning

2) **UNKNOWN_ASSUMPTIONS** (unknown_assumptions.v1)
- explicit unknowns and assumptions list
- deterministic IDs and pointers into canonical/spec

Optional artifacts (recommended)
3) **CANONICAL_BUILD_REPORT** (canonical_build_report.v1)
- a concise report of:
  - how many entities by type
  - how many relationships by type
  - unknowns count by severity
  - any integrity check failures/warnings (if non-fatal in prototype)

Piece B — Canonical build report schema

File:
`Axion/libraries/canonical/schemas/canonical_build_report.v1.schema.jso
n`

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "schema_version": "1.0.0",
  "id": "canonical_build_report.v1",
  "type": "object",
  "required": ["run_id", "spec_ref", "ua_ref", "counts", "generated_at"],
  "properties": {
    "run_id": { "type": "string" },

    "spec_ref": {
      "type": "object",
      "required": ["path"],
      "properties": { "path": { "type": "string" }, "hash": { "type": "string" } },
      "additionalProperties": false
    },

    "ua_ref": {
      "type": "object",
      "required": ["path"],
      "properties": { "path": { "type": "string" }, "hash": { "type": "string" } },
      "additionalProperties": false
    },

    "counts": {
      "type": "object",
      "required": ["entities_by_type", "relationships_by_type", "unknowns_by_severity"],
      "properties": {
        "entities_by_type": { "type": "object", "additionalProperties": { "type": "integer", "minimum": 0
} },
        "relationships_by_type": { "type": "object", "additionalProperties": { "type": "integer",
"minimum": 0 } },
        "unknowns_by_severity": { "type": "object", "additionalProperties": { "type": "integer",
"minimum": 0 } }
      },
      "additionalProperties": false
    },

    "notes": { "type": "array", "items": { "type": "string" } },

    "generated_at": { "type": "string", "format": "date-time" }
```

```
  },
  "additionalProperties": false
}
```

Piece C — Artifact registration requirements (run manifest pointers)

File: `Axion/libraries/canonical/CAN-5_manifest_requirements.md`

```
---
library: canonical
id: CAN-5a
schema_version: 1.0.0
status: draft
---

# CAN-5a — Manifest Requirements (Canonical Outputs)

When S4 completes successfully, the run manifest must include artifact entries for:

- contract_id: CANONICAL_SPEC
- contract_id: UNKNOWN_ASSUMPTIONS
- (optional) contract_id: CANONICAL_BUILD_REPORT

Each artifact entry must include:
- path
- produced_by_stage: S4_CANONICAL_BUILD
- hash if hashing enabled
```

Piece D — Determinism rules

File: `Axion/libraries/canonical/CAN-5_determinism_rules.md`

```
---
library: canonical
id: CAN-5b
schema_version: 1.0.0
status: draft
---

# CAN-5b — Determinism Rules (Artifacts)
```

- Canonical spec content is deterministic (CAN-1..CAN-3).
- Unknowns artifact content is deterministic (CAN-4).
- Build report counts are derived from those pinned artifacts only.
- Notes in build report must be stable (avoid timestamps, random phrasing).

Piece E — Validation checklist

File: `Axion/libraries/canonical/CAN-5_validation_checklist.md`

```
---
library: canonical
id: CAN-5c
schema_version: 1.0.0
status: draft
---

# CAN-5c — Validation Checklist

- [ ] canonical_spec validates against schema
- [ ] unknown_assumptions validates against schema
- [ ] canonical_build_report validates against schema (if used)
- [ ] run manifest records required artifacts with correct contract_ids
- [ ] artifact produced_by_stage == S4_CANONICAL_BUILD
```

Say **“next”** for **CAN-6 Canonical gates (integrity, completeness thresholds)**.

Canonical gates

CAN-6 — Canonical gates (integrity, completeness thresholds)

Piece A — Canonical gate intent

File: `Axion/libraries/canonical/CAN-6_canonical_gates.md`

```
---
library: canonical
id: CAN-6
schema_version: 1.0.0
status: draft
---

# CAN-6 — Canonical Gates

## Purpose
Enforce that the Canonical Spec is internally consistent and usable by downstream stages.

## Gate set (minimum)

### CAN-GATE-01 — Canonical spec schema valid
- CANONICAL_SPEC validates against canonical_spec.v1

### CAN-GATE-02 — ID integrity
- entity_id unique
- rel_id unique
- IDs follow patterns and are stable per CAN-2 rules (where enforceable)

### CAN-GATE-03 — Relationship integrity
- all relationships from/to exist in entities (CAN-INTEGRITY-01)

### CAN-GATE-04 — Relationship type compatibility
- relationship types comply with relationship_constraints registry (CAN-INTEGRITY-02)

### CAN-GATE-05 — Uniqueness constraints
- endpoints unique by (method + path)
- components unique by name_slug (project scope)
- data_entities unique by name_slug (project scope)

### CAN-GATE-06 — Unknowns handling
- UNKNOWN_ASSUMPTIONS exists
```

- blocking unknowns behavior is risk-class dependent:
  - PROTOTYPE: allowed but recorded
  - PROD/COMPLIANCE: may block unless overridden (policy set decides)

Output
- Canonical build report (optional) should summarize counts and unknown severity.

Piece B — Mapping to pipeline gate (G2_CANONICAL_INTEGRITY)

File: `Axion/libraries/canonical/CAN-6_gate_mapping.md`

---
library: canonical
id: CAN-6a
schema_version: 1.0.0
status: draft

CAN-6a — Mapping to Pipeline Gates

Pipeline gate: **G2_CANONICAL_INTEGRITY**

G2 passes if:
- CAN-GATE-01..05 pass
- CAN-GATE-06 satisfies risk-class policy (blocking unknowns handled)

Risk-class rule (default):
- PROTOTYPE: blocking unknowns produce warn/pause (configurable)
- PROD: blocking unknowns → pause until resolved or approved override
- COMPLIANCE: blocking unknowns → hard stop (default)

Piece C — Machine-readable gate spec (optional)

File: `Axion/libraries/canonical/CAN-6_canonical_gates.spec.json`

```
{
  "schema_version": "1.0.0",
  "contract_id": "CAN-6b",
  "gates": [
```

{ "gate_id": "CAN-GATE-01", "name": "Canonical spec schema valid", "severity": "hard_stop", "checks": ["schema_valid(canonical_spec.v1)"] },
{ "gate_id": "CAN-GATE-02", "name": "ID integrity", "severity": "hard_stop", "checks": ["unique(entity_id)", "unique(rel_id)"] },
{ "gate_id": "CAN-GATE-03", "name": "Relationship integrity", "severity": "hard_stop", "checks": ["all relationships.from/to exist"] },
{ "gate_id": "CAN-GATE-04", "name": "Relationship type compatibility", "severity": "hard_stop", "checks": ["relationship_constraints respected"] },
{ "gate_id": "CAN-GATE-05", "name": "Uniqueness constraints", "severity": "hard_stop", "checks": ["unique endpoint(method+path)", "unique component(name_slug)", "unique data_entity(name_slug)"] },
{ "gate_id": "CAN-GATE-06", "name": "Unknowns handling", "severity": "policy_controlled", "checks": ["UNKNOWN_ASSUMPTIONS exists", "blocking unknown behavior per risk class"] }
]
}

Piece D — Evidence requirements

File: `Axion/libraries/canonical/CAN-6_evidence_requirements.md`

```
---
library: canonical
id: CAN-6c
schema_version: 1.0.0
status: draft
---

# CAN-6c — Evidence Requirements (Canonical Failures)

For any failure, evidence must include:
- failing gate_id
- entity_id/rel_id list (when applicable)
- pointers to canonical_spec locations (JSON pointers)
- pointers to unknown_assumptions entries (when applicable)
- remediation:
  - fix builder rules
  - resolve unknown via intake update
  - rerun S4 and downstream stages
```

Piece E — Determinism rules

File: `Axion/libraries/canonical/CAN-6_determinism_rules.md`

```
---
library: canonical
id: CAN-6d
schema_version: 1.0.0
status: draft
---

# CAN-6d — Determinism Rules (Canonical Gates)

- Canonical gates run in fixed order CAN-GATE-01..06.
- Failure lists are deterministically ordered (entity_id/rel_id sorted).
- Policy-controlled behavior must be based on pinned risk_class + policy_set decision.
```

Piece F — Validation checklist

File: `Axion/libraries/canonical/CAN-6_validation_checklist.md`

```
---
library: canonical
id: CAN-6e
schema_version: 1.0.0
status: draft
---

# CAN-6e — Validation Checklist

- [ ] Canonical gate set covers schema, IDs, relationships, type constraints, uniqueness, unknowns
- [ ] G2 mapping is explicit
- [ ] Policy-controlled unknown behavior is defined and deterministic
- [ ] Evidence pointers are included for actionable remediation
```

Say **“next”** for **CAN-7 Minimum viable canonical set**.

Minimum viable canonical

CAN-7 — Minimum viable canonical set (runtime-required files + “done”)

Piece A — MV canonical inventory

File: `Axion/libraries/canonical/CAN-7_minimum_viable_set.md`

---
library: canonical
id: CAN-7
schema_version: 1.0.0
status: draft

CAN-7 — Minimum Viable canonical/ Set

Goal
Define the smallest canonical library set required to build a usable canonical spec deterministically.

Required runtime files (minimum)
1) Core docs
- CAN-0_purpose.md
- CAN-0_boundary_checklist.md
- CAN-1_entity_model.md
- CAN-1_determinism_rules.md
- CAN-1_validation_checklist.md
- CAN-2_id_rules.md
- CAN-2_id_generation_spec.md
- CAN-2_dedupe_rules.md
- CAN-2_determinism_rules.md
- CAN-2_validation_checklist.md
- CAN-3_reference_integrity.md
- CAN-3_integrity_checks.md
- CAN-3_determinism_rules.md
- CAN-3_validation_checklist.md
- CAN-4_unknowns_assumptions_model.md
- CAN-4_rules.md
- CAN-4_determinism_rules.md
- CAN-4_validation_checklist.md
- CAN-5_artifacts.md
- CAN-5_manifest_requirements.md

- CAN-5_determinism_rules.md
- CAN-5_validation_checklist.md
- CAN-6_canonical_gates.md
- CAN-6_gate_mapping.md
- CAN-6_evidence_requirements.md
- CAN-6_determinism_rules.md
- CAN-6_validation_checklist.md
- CAN-7_minimum_viable_set.md

2) Schemas (must be in schema_registry)
- schemas/canonical_spec.v1.schema.json
- schemas/unknown_assumptions.v1.schema.json
- schemas/canonical_build_report.v1.schema.json

3) Registries (runtime defaults)
- registries/id_rules.v1.json (optional but recommended)
- registries/relationship_constraints.v1.json

4) Gate specs (optional)
- CAN-6_canonical_gates.spec.json

Piece B — Definition of done (canonical/)

File: `Axion/libraries/canonical/CAN-7_definition_of_done.md`

---
library: canonical
id: CAN-7a
schema_version: 1.0.0
status: draft

CAN-7a — canonical/ Definition of Done

canonical/ is “done” when:

Schemas + registries
- [ ] canonical_spec schema validates
- [ ] unknown_assumptions schema validates
- [ ] canonical_build_report schema validates
- [ ] relationship_constraints registry exists and validates
- [ ] id rules are pinned (if used)

Runtime behavior (contract-level)
- [ ] S4 produces CANONICAL_SPEC and UNKNOWN_ASSUMPTIONS deterministically
- [ ] IDs are stable across reruns (no random IDs)
- [ ] Relationship integrity enforced (no dangling refs)
- [ ] Type compatibility enforced via relationship_constraints
- [ ] Dedupe conflicts become unknown/assumption items (no silent overwrite)

Gates
- [ ] CAN-GATE-01..06 implemented and mapped to G2_CANONICAL_INTEGRITY
- [ ] Blocking unknown behavior follows risk class + policy decisions
- [ ] Failures include actionable pointers and remediation steps

---

Piece C — Minimal folder tree

File: `Axion/libraries/canonical/CAN-7_minimal_tree.txt`

```
Axion/libraries/canonical/
  CAN-0_purpose.md
  CAN-0_boundary_checklist.md
  CAN-1_entity_model.md
  CAN-1_determinism_rules.md
  CAN-1_validation_checklist.md
  CAN-2_id_rules.md
  CAN-2_id_generation_spec.md
  CAN-2_dedupe_rules.md
  CAN-2_determinism_rules.md
  CAN-2_validation_checklist.md
  CAN-3_reference_integrity.md
  CAN-3_integrity_checks.md
  CAN-3_determinism_rules.md
  CAN-3_validation_checklist.md
  CAN-4_unknowns_assumptions_model.md
  CAN-4_rules.md
  CAN-4_determinism_rules.md
  CAN-4_validation_checklist.md
  CAN-5_artifacts.md
  CAN-5_manifest_requirements.md
  CAN-5_determinism_rules.md
  CAN-5_validation_checklist.md
  CAN-6_canonical_gates.md
  CAN-6_gate_mapping.md
```

```
CAN-6_evidence_requirements.md
CAN-6_determinism_rules.md
CAN-6_validation_checklist.md
CAN-7_minimum_viable_set.md
CAN-7_definition_of_done.md
CAN-7_minimal_tree.txt

schemas/
  canonical_spec.v1.schema.json
  unknown_assumptions.v1.schema.json
  canonical_build_report.v1.schema.json

registries/
  relationship_constraints.v1.json
  id_rules.v1.json
```